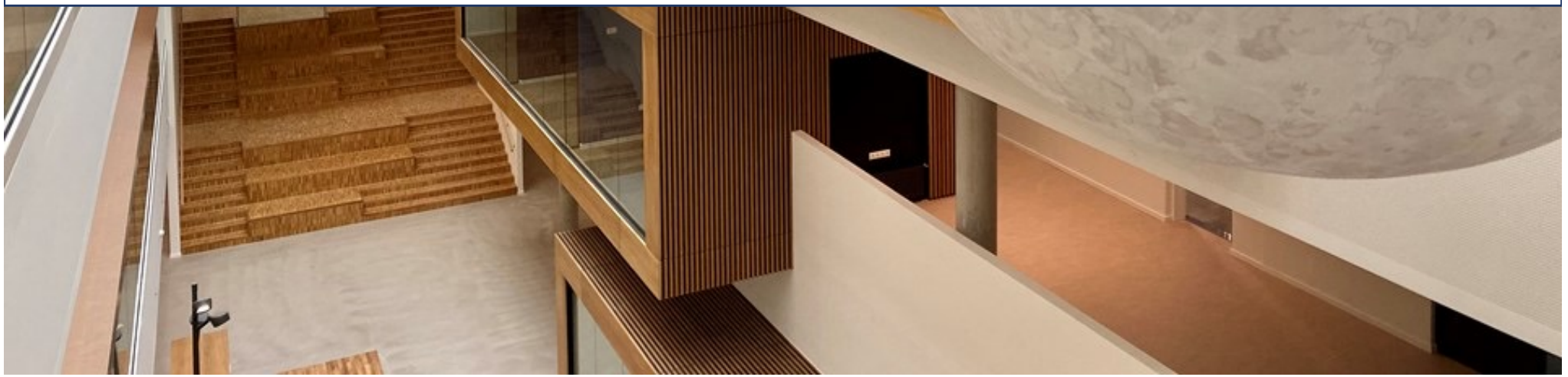


Introduction to AI

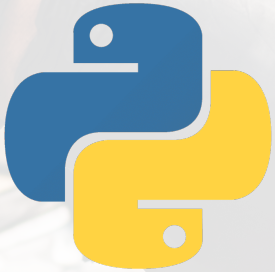
MM2: Python for machine learning and kNN

Thomas Kronborg Larsen, PhD
(tkl@hst.aau.dk)



Let's get everything ready…

Programming language:



python™

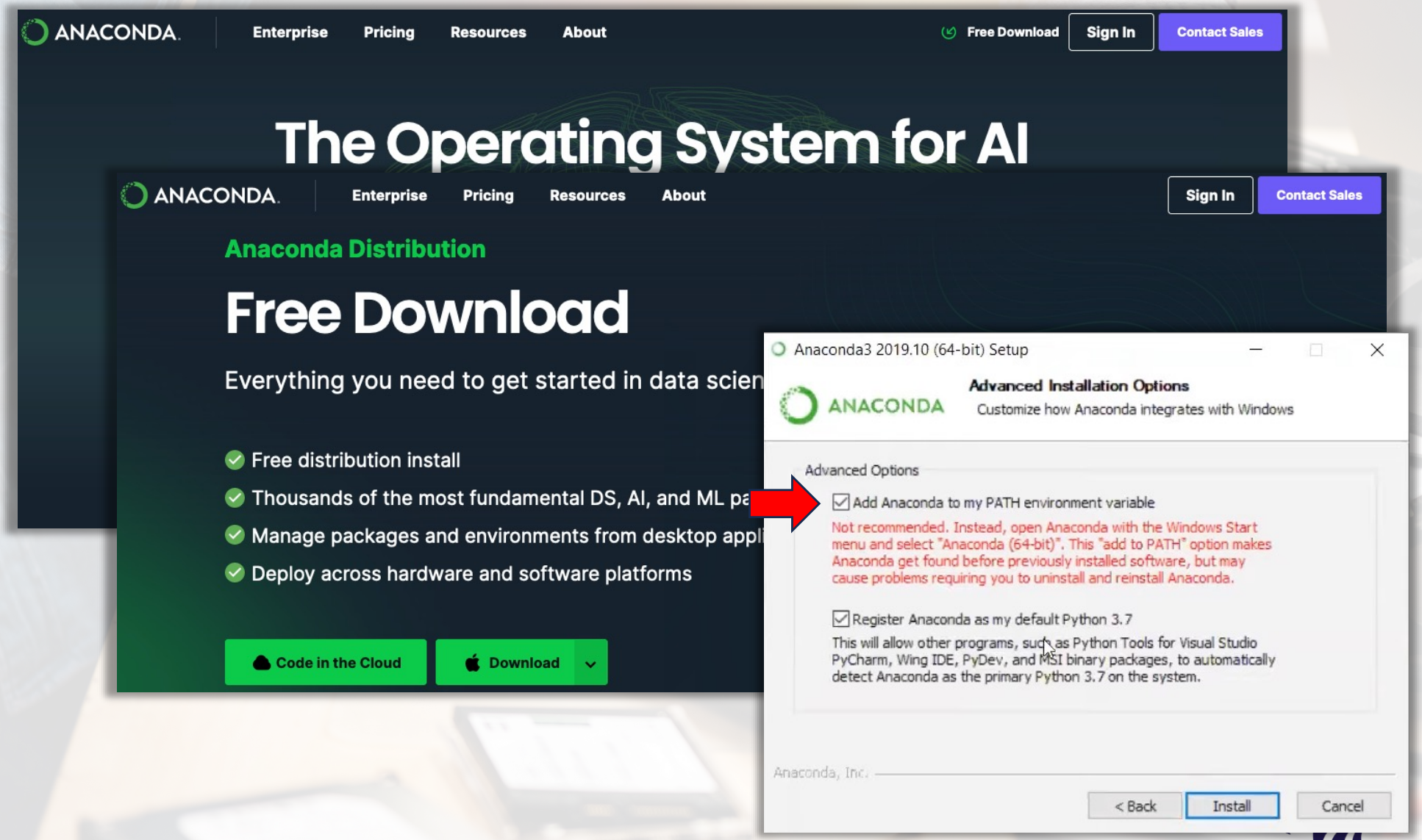
Our "workbench" with tools:



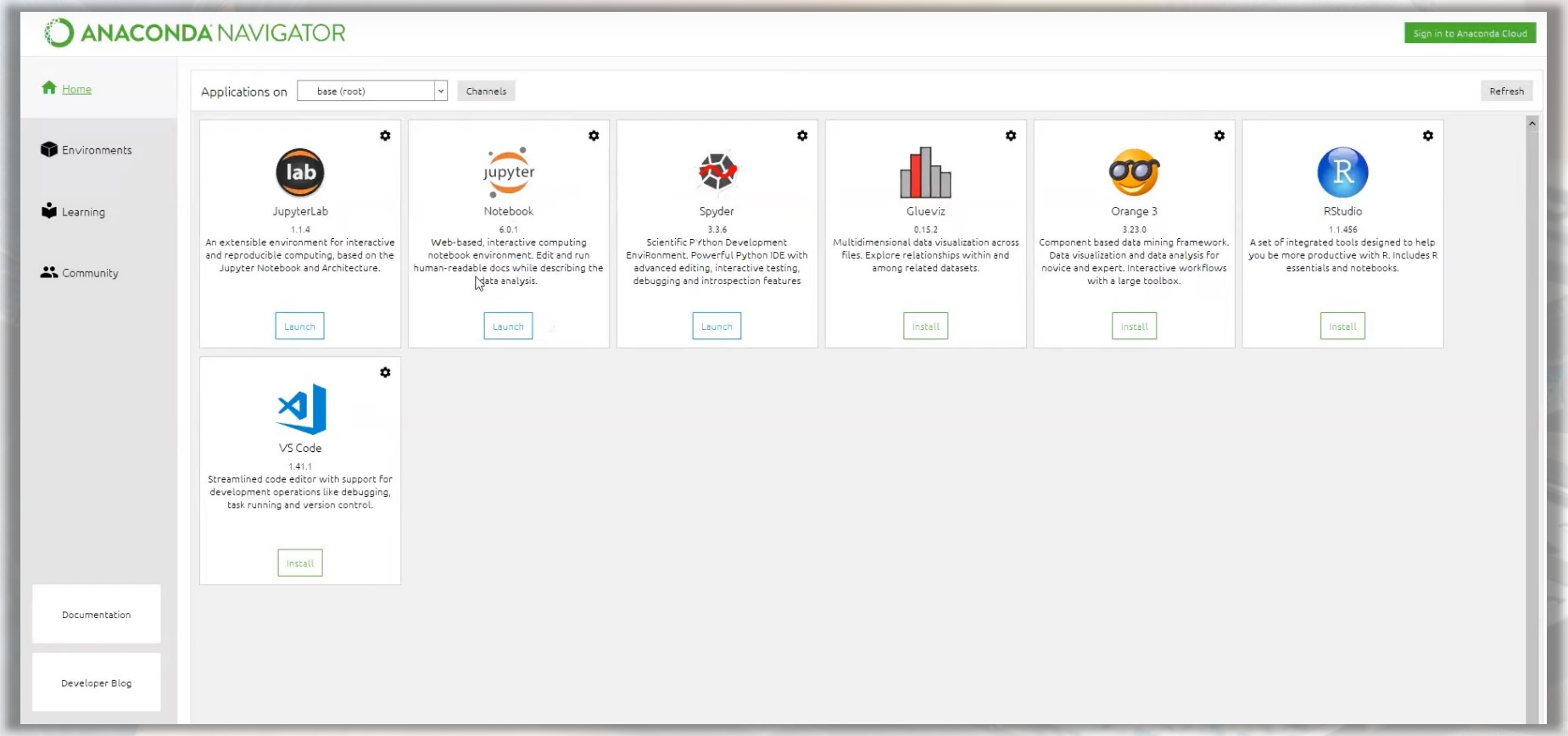
ANACONDA®

Install Anaconda

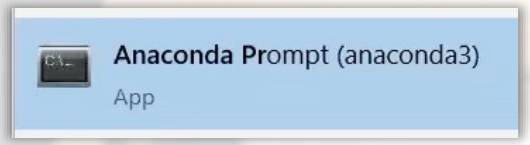
https://www.anaconda.com/



Search and open Anaconda Navigator



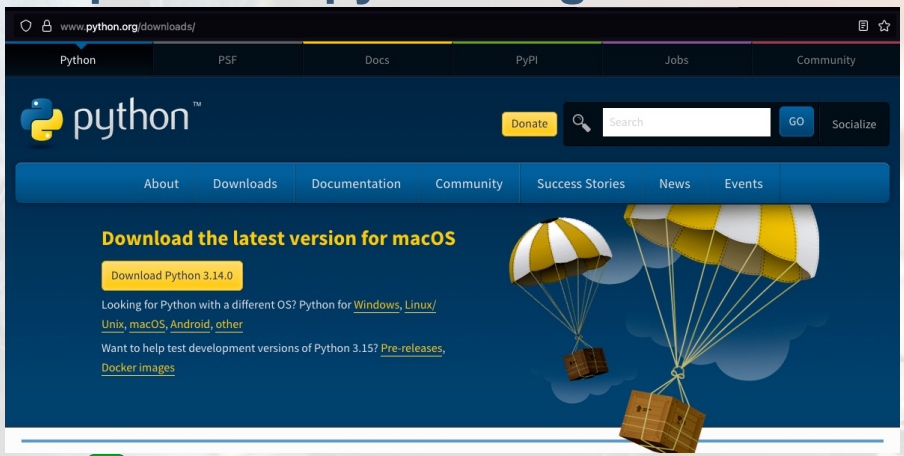
Install additional packages



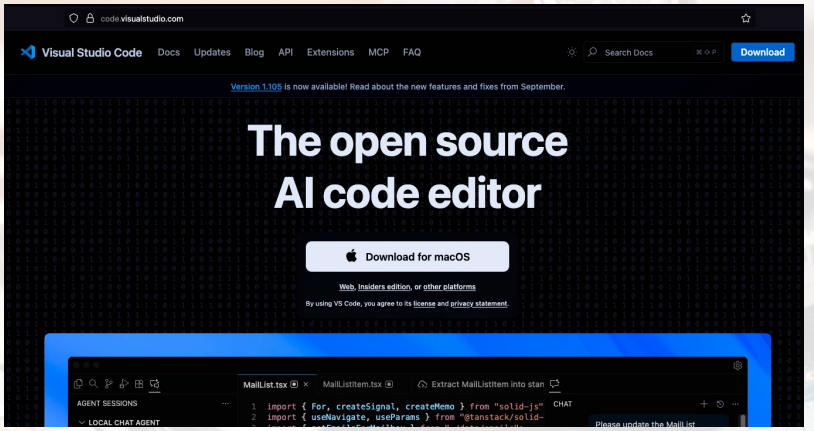
Type: pip install <package name>

If you cannot install Anaconda

https://www.python.org/downloads/



https://code.visualstudio.com/



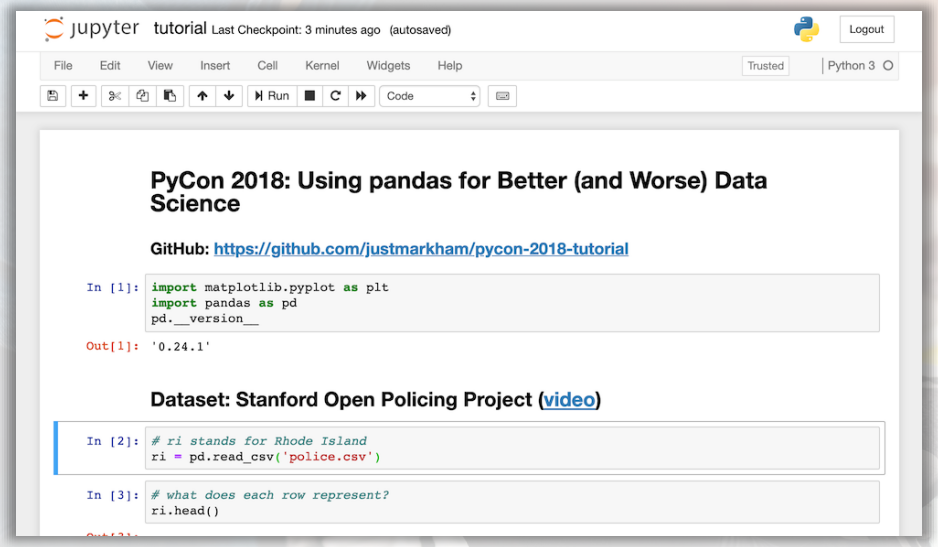
✅ **"Add Python to PATH"** (during installation)

In the VS Code terminal

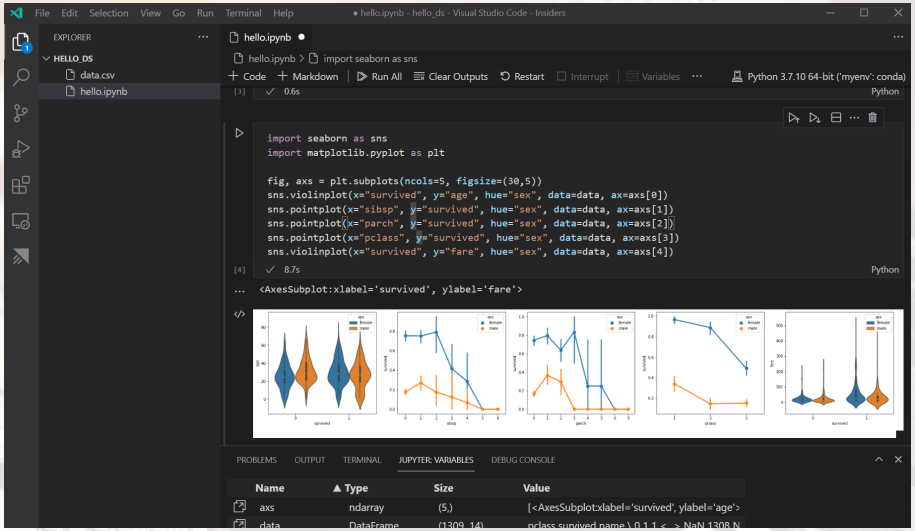
Step	Action	Command
1	Check Python	`python --version`
2	Create virtual env	`python -m venv venv`
3	Activate env	`venv\Scripts\activate` (Windows) or `source venv/bin/activate` (macOS)
4	Install packages	`pip install pandas numpy scikit-learn matplotlib seaborn statsmodels jupyter`
5	Register kernel	`python -m ipykernel install --user --name=venv --display-name "Python (venv)"`
6	Open VSCode file and select kernel	

IDE Integrated Development Environment

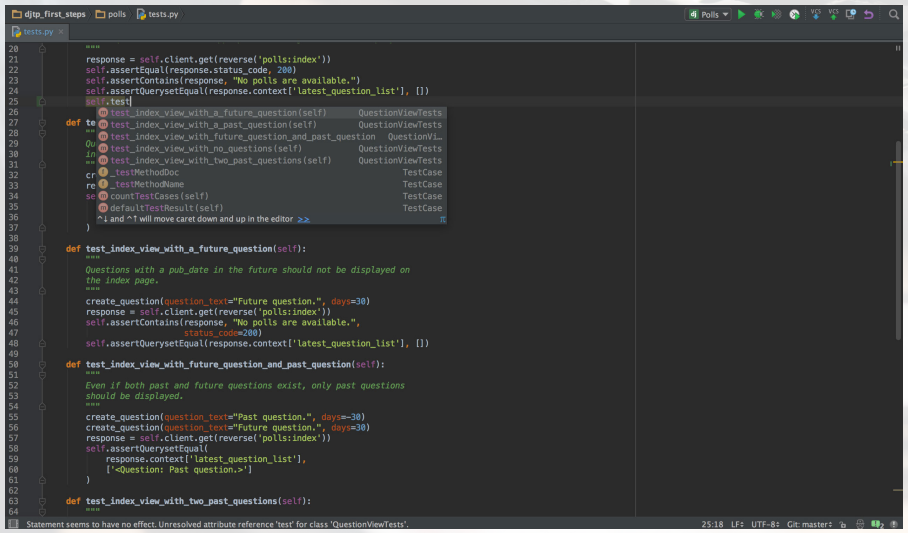
Jupyter notebook



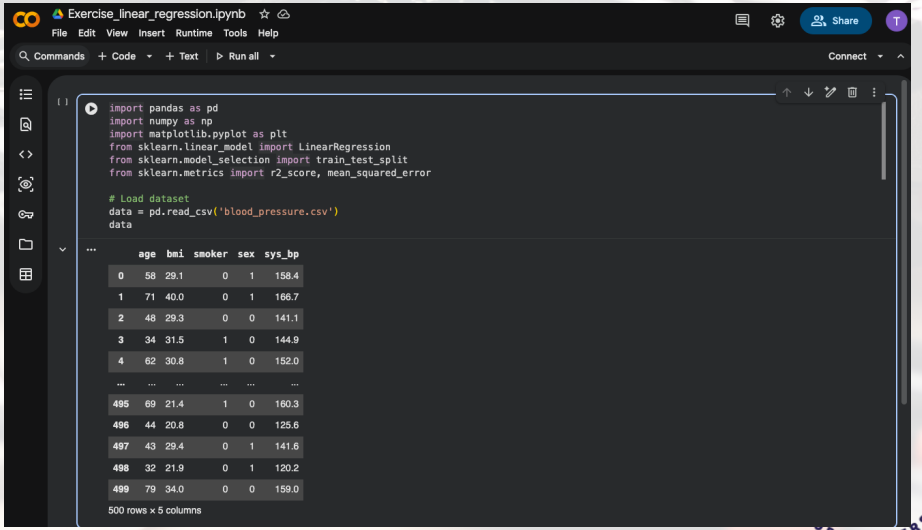
Visual studio code



PyCharm



Google colab



Virtual environments



An essential tool for:

- Managing dependencies
- Ensuring reproducibility
- Maintaining a clean and organized development environment
- Conflict Resolution
- Experimentation

Creating the environment through conda (command promt or terminal):

- conda create -n venv python=3.12
- conda activate venv
- conda install numpy pandas matplotlib scikit-learn jupyter ipykernel
- python -m ipykernel install --user --name venv --display-name "Python (venv)"

AD NYE VEJE
AALBORG UNIVERSITET

Let’s open Jupyter Notebook and test it out:

```python
# What version of Python and packages do you have?
import sys

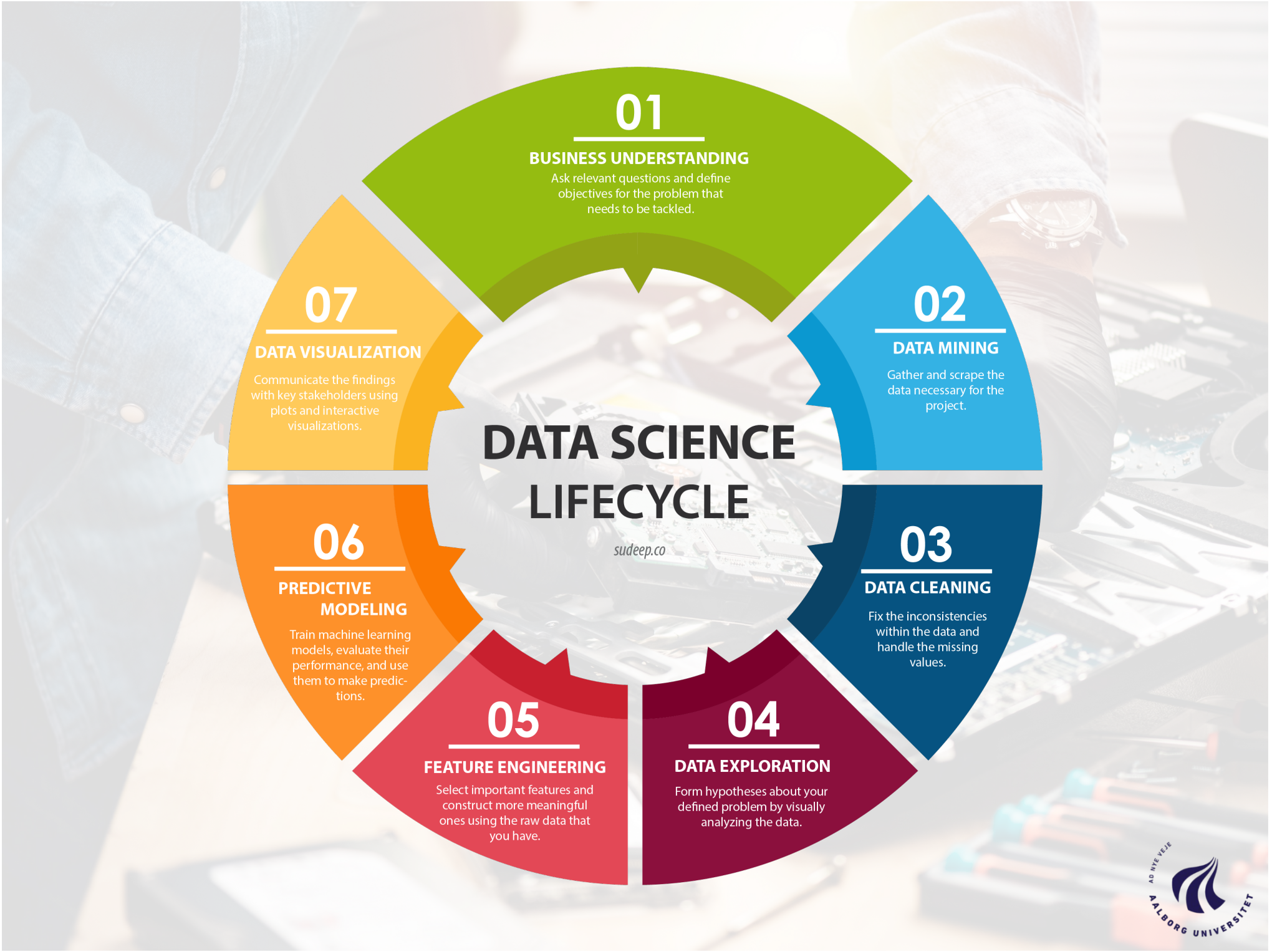
import pandas as pd
import sklearn as sk
import matplotlib as mpl

print(f"Python {sys.version}")
print(f"Pandas {pd.__version__}")
print(f"Scikit-Learn {sk.__version__}")
print(f"Matplotlib {mpl.__version__}")
```

```
Python 3.11.5 (main, Sep 11 2023, 08:19:27) [Clang 14.0.6 ]
Pandas 2.0.3
Scikit-Learn 1.3.0
Matplotlib 3.7.2
```



AALBORG UNIVERSITET


01
BUSINESS UNDERSTANDING
Ask relevant questions and define objectives for the problem that needs to be tackled.
02
DATA MINING
Gather and scrape the data necessary for the project.
03
DATA CLEANING
Fix the inconsistencies within the data and handle the missing values.
04
DATA EXPLORATION
Form hypotheses about your defined problem by visually analyzing the data.
05
FEATURE ENGINEERING
Select important features and construct more meaningful ones using the raw data that you have.
06
PREDICTIVE MODELING
Train machine learning models, evaluate their performance, and use them to make predictions.
07
DATA VISUALIZATION
Communicate the findings with key stakeholders using plots and interactive visualizations.
DATA SCIENCE LIFECYCLE
sudeep.co
AD NYE VEJE
AALBORG UNIVERSITET

Data operations using pandas

```python
import pandas as pd

# 1. Read Data
df = pd.read_csv('data.csv')

# 2. Head & Tail
df.head() # First 5 rows
df.tail() # Last 5 rows

# 3. Info
df.info()

# 4. Descriptive statistics
df.describe()

# 5. Handling Missing Values
df_dropna = df.dropna()

df_fillna = df.fillna(0)  # Filling missing values with 0
```

AALBORG UNIVERSITET

Data operations using pandas

```python
# 6. Selecting Columns
selected_columns = df[['column1', 'column2']]

# 7. Filtering
filtered_df = df[df['column'] > 10]

# 8. Group By
grouped_df = df.groupby('column').agg({'column2': 'mean'})

# 9. Merge & Concatenate
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'A': [7, 8, 9], 'B': [10, 11, 12]})

merged_df = pd.merge(df1, df2, on='A')

# 10. Applying Functions
df['new_column'] = df['column'].apply(lambda x: x * 2)
```

Data operations using pandas

Data Wrangling

with pandas Cheat Sheet

http://pandas.pydata.org

Pandas API Reference Pandas User Guide

Tidy Data – A foundation for wrangling in pandas

In a tidy data set:

Each **variable** is saved in its own **column**

Each **observation** is saved in its own **row**

Tidy data complements pandas's vectorized operations. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
        {"a" : [4, 5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
       index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

		a	b	c
N	v			
D	1	4	7	10
	2	5	8	11
e	2	6	9	12

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
index = pd.MultiIndex.from_tuples(
        [('d', 1), ('d', 2),
         ('e', 2)], names=['n', 'v']))
```

Create DataFrame with a MultiIndex

Method Chaining

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
        .rename(columns={
                'variable':'var',
```

Reshaping Data – Change layout, sorting, reindexing, renaming

`pd.melt(df)`
Gather columns into rows.

`df.pivot(columns='var`
Spread rows into columns.

`pd.concat([df1,df2])`
Append rows of DataFrames

`pd.concat([df1,df2],`
Append columns of DataFra

`df.sort_values('mpg')`
Order rows by values of a column (low to high).

`df.sort_values('mpg', ascending=False)`



Subset Observations - rows

```
df[df.Length > 7]
```
Extract rows that meet logical criteria.

`df.drop_duplicates()`
Remove duplicate rows (only considers columns).

`df.sample(frac=0.5)`
Randomly select fraction of rows.

`df.sample(n=10)` Randomly select n rows.

`df.nlargest(n, 'value')`
Select and order top n entries.

`df.nsmallest(n, 'value')`
Select and order bottom n entries.

`df.head(n)`
Select first n rows.

`df.tail(n)`
Select last n rows.

Subset Variab

```
df[['width', 'lengt
```
Select multiple column

`df['width']` *or* `df`
Select single column w

`df.filter(regex='regex')`
Select columns whose name matches regular expression *regex*.

Using query

query() allows Boolean expressions for filtering rows.

```
df.query('Length > 7')
df.query('Length > 7 and Width < 8')
df.query('Name.str.startswith("abc")',
         engine="python")
```

`df.iloc[:, [1, 2, 5]]`
Select columns in positions 1, 2 and 5 (first column is 0).

`df.loc[:, 'x2':'x4']`
Select all columns between x2 and x4 (inclusive).

`df.loc[df['a'] > 10, ['a', 'c']]`
Select rows meeting logical condition, and only the specific columns .

`df.iat[1, 2]` Access single value by index

`df.at[4, 'A']` Access single value by label

Logic in Python (and pandas)			
<	Less than	!=	Not equal to
>	Greater than	`df.column.isin(values)`	Group membership
==	Equals	`pd.isnull(obj)`	Is NaN

regex (Regular Expressions) Examples	
'\.'	Matches strings containing a period '.'
'Length$'	Matches strings ending with word 'Length'
'^Sepal'	Matches strings beginning with the word 'Sepal'

AALBORG UNIVERSITET

Machine learning using Scikit-learn

StandardScaler: Standardize features by removing the mean and scaling to unit

OneHotEncoder: Encode categorical integer features as one-hot numeric arrays.

Dimensionality reduction: Reduce number of features e.g. using PCA and LDA

Models:

- Naïve bayes
- Linear regression
- Logistic regression
- Random forest
- Etc.

Performance metrics: Evaluation the model like accuracy, sensitivity, specificity, etc.

AD NYE VEJE
AALBORG UNIVERSITET

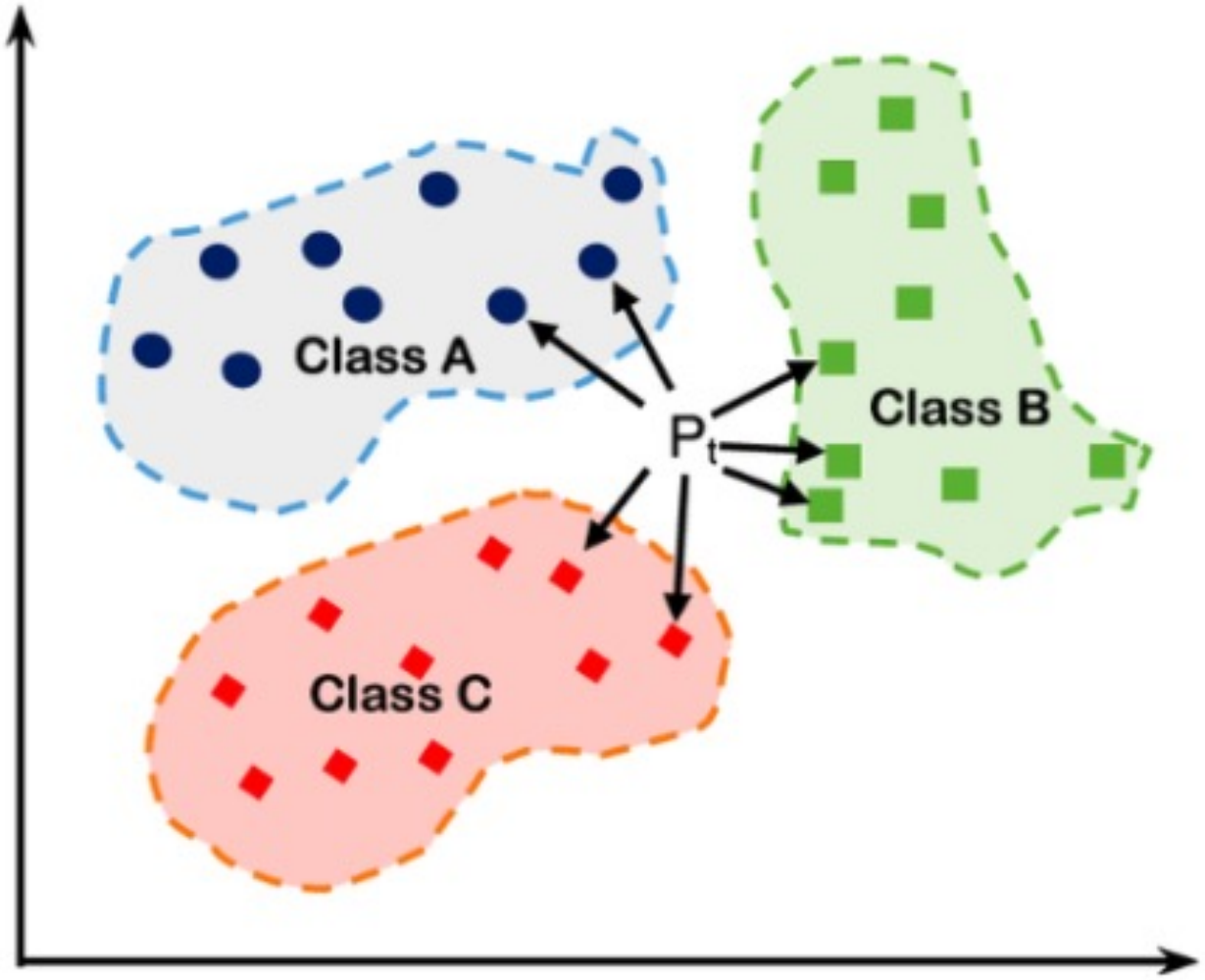
Test your knowledge



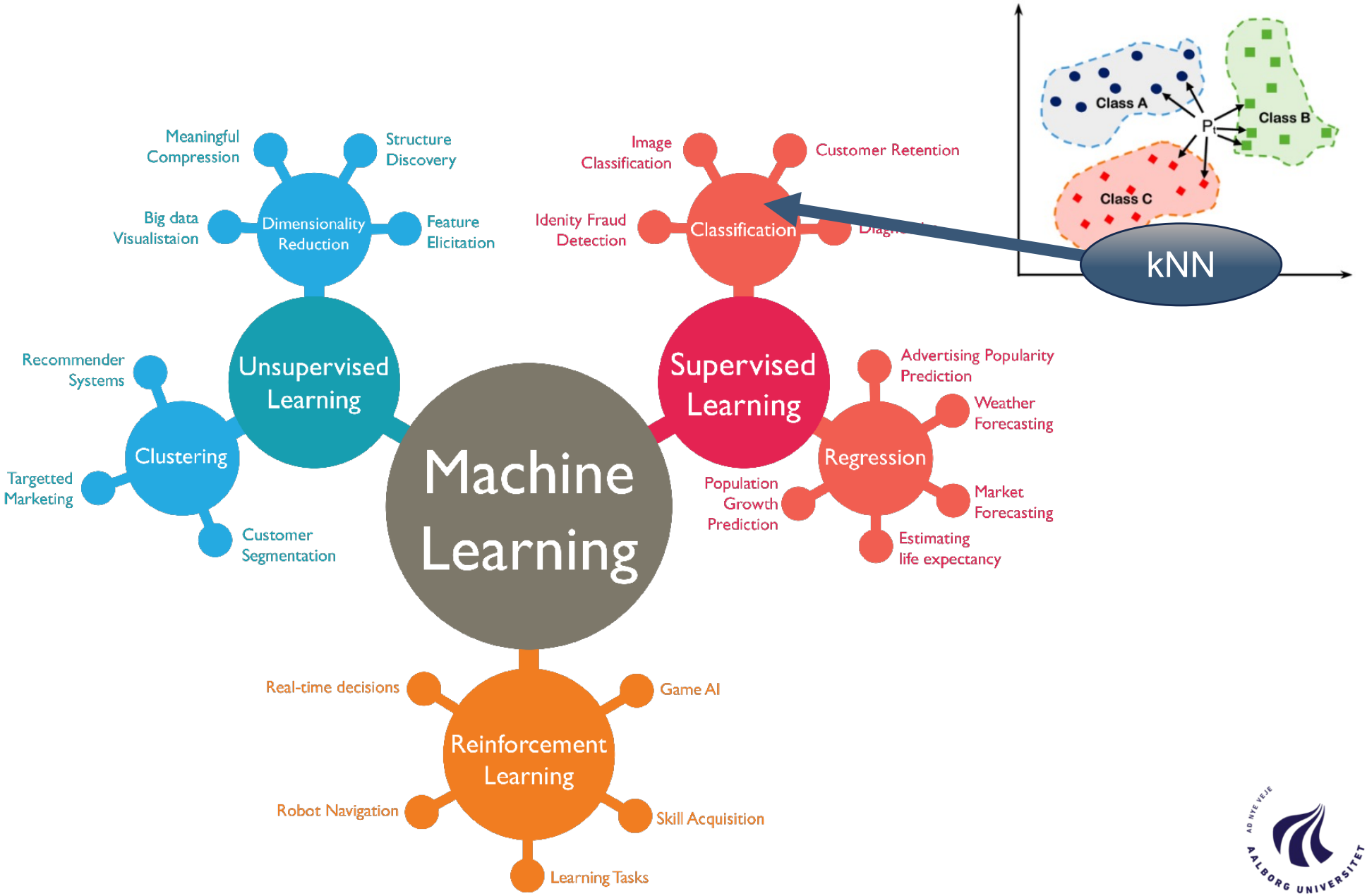
1. What is the difference between Python and Anaconda?
2. What is a virtual environment - and why should we use one?
3. When do we use "*conda install*" in the terminal (mac) or command promt (windows)?
4. What does it mean to import a library?
5. What is the likely problem if we run "import pandas as pd" and interpreter states "ModuleNotFoundError: No module named pandas"?
6. When and why do we use feature encoding?
7. Why do we scale features?

AD NYE VEJE
AALBORG UNIVERSITET

K-Nearest Neighbor



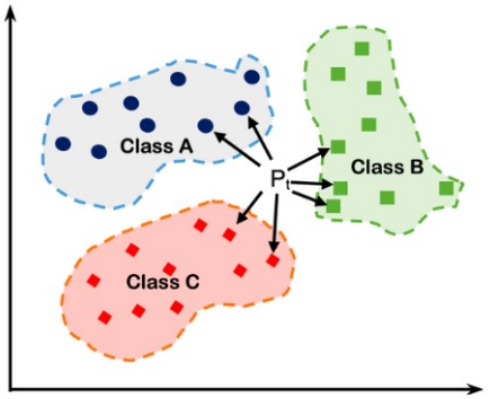
K-Nearest Neighbor



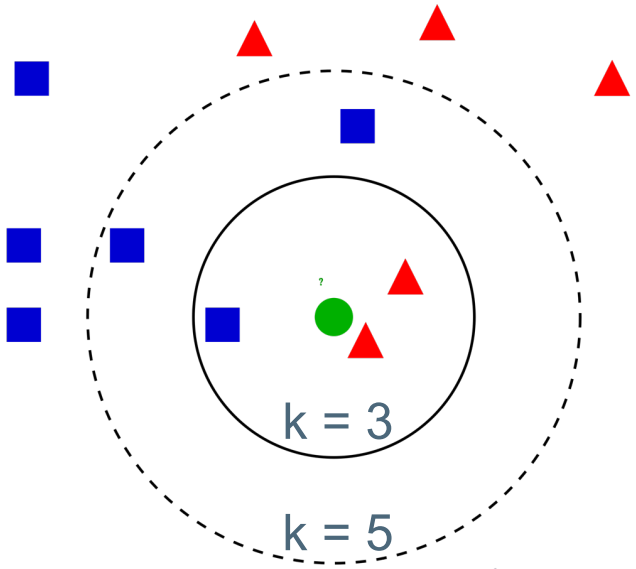
K-Nearest Neighbor

How does the algorithm work?

1. Select k (number of neighbors)
2. Find k nearest labeled training samples (**x'**) around the test sample (**x**)
3. Classify **x** as the most frequent class label in the k-nearest samples



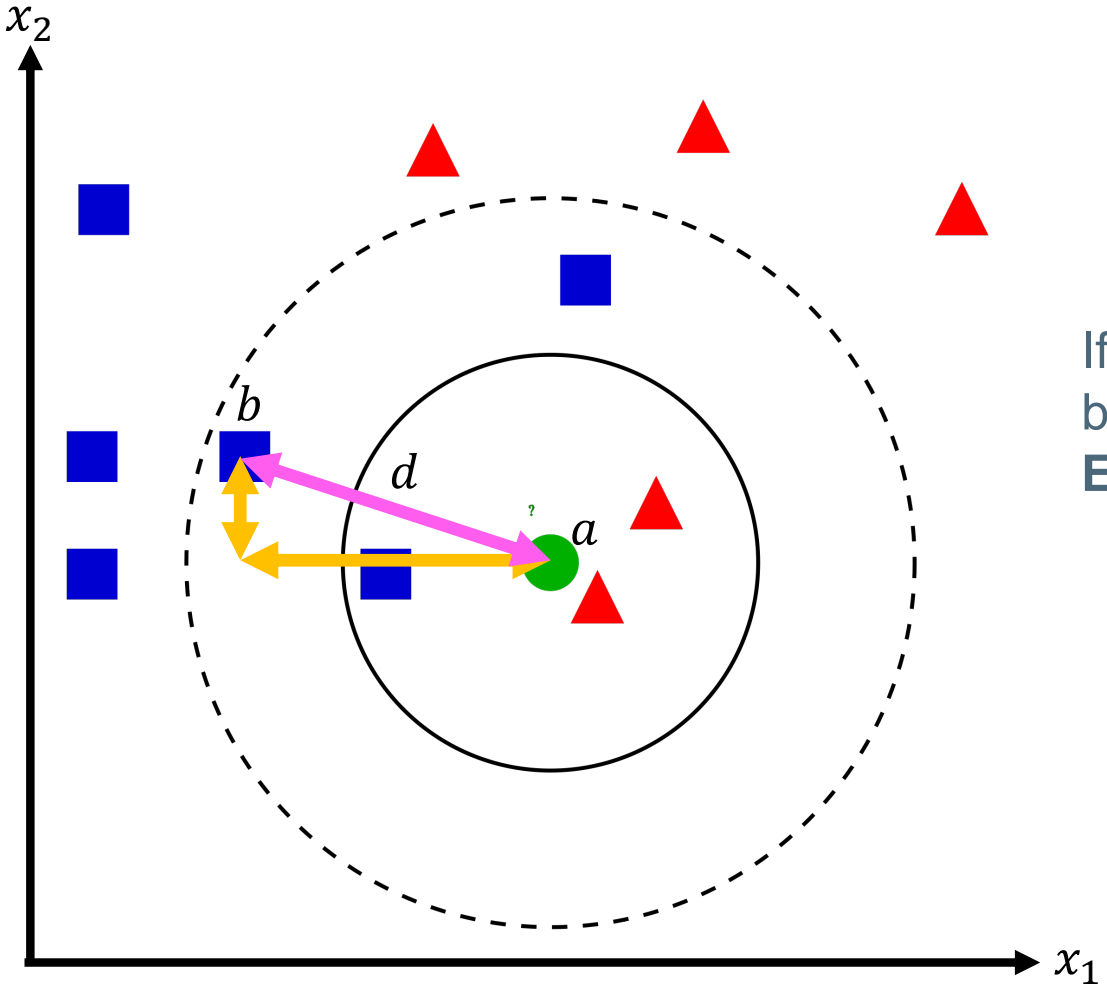
Example

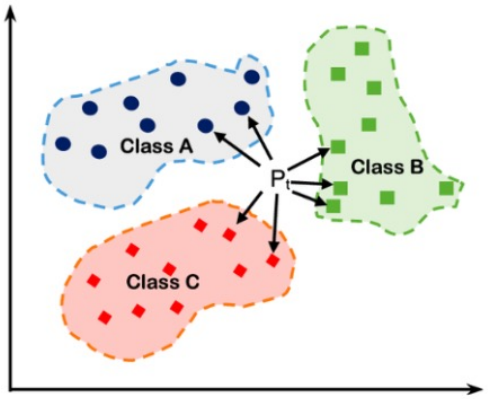


k should be uneven to prevent ties (Can be trained)
k is often equal to $\sqrt{n}$, where n is the number of training samples
Feature scaling is important because we use distance!

K-Nearest Neighbor

How do we measure distance?





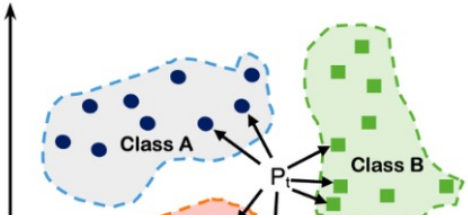
If we want to measure the distance between point a and point b, we use **Euclidian distance:**

$$d(a,b) = \sqrt{\sum_{i=1}^{n}(a_i - b_i)^2}$$

(Hence, feature scaling is important)

K-Nearest Neighbor

Using **Scikit-Learn**:




```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # Using only the first two features (sepal length and sepal width)
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a kNN classifier with a specified value of k (e.g., k=3)
k = 3
knn_classifier = KNeighborsClassifier(n_neighbors=k)

# Fit the classifier to the training data
knn_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = knn_classifier.predict(X_test)

# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of kNN with k={k}: {accuracy * 100:.2f}%")
```

```
Accuracy of kNN with k=3: 75.56%
```

AALBORG UNIVERSITET

K-Nearest Neighbor

Advantages

- **Simple and easy to implement**:
- **Adapts easily**: As new training samples are added, the algorithm adjusts to account for any new data since all training data is stored into memory
- **Few hyperparameters**: KNN only requires a k value

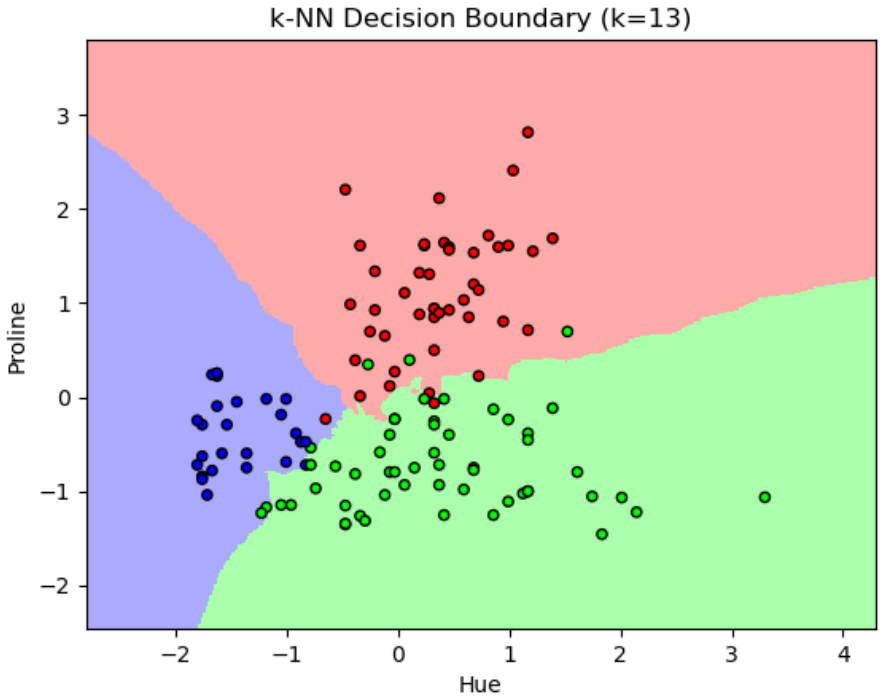
Disadvantages

- **Does not scale well**: Being a “lazy” algorithm, it takes up more memory and data
  Curse of dimensionality: The KNN algorithm tends to fall victim to the curse of dimensionality, which means that it doesn’t perform well with high-dimensional data inputs.
- **Prone to overfitting**: Lower values of k can overfit the data, whereas higher values of k tend to “smooth out” the prediction

AD NYE VEJE
AALBORG UNIVERSITET

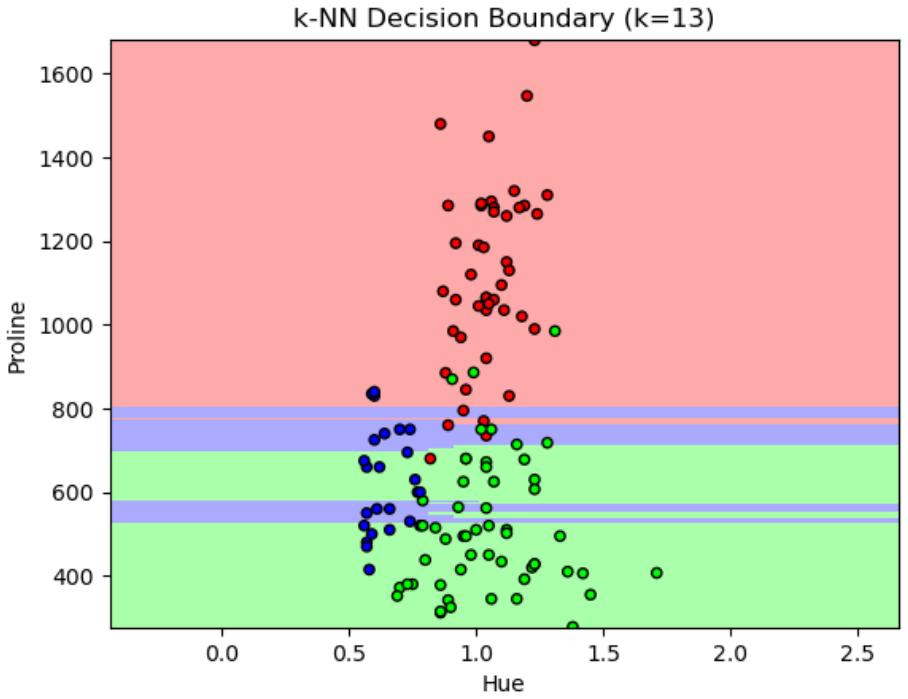
K-Nearest Neighbor

With feature scaling



Accuracy on training data: 0.90
Accuracy on test data: 0.81

Without feature scaling



Accuracy on training data: 0.75
Accuracy on test data: 0.59

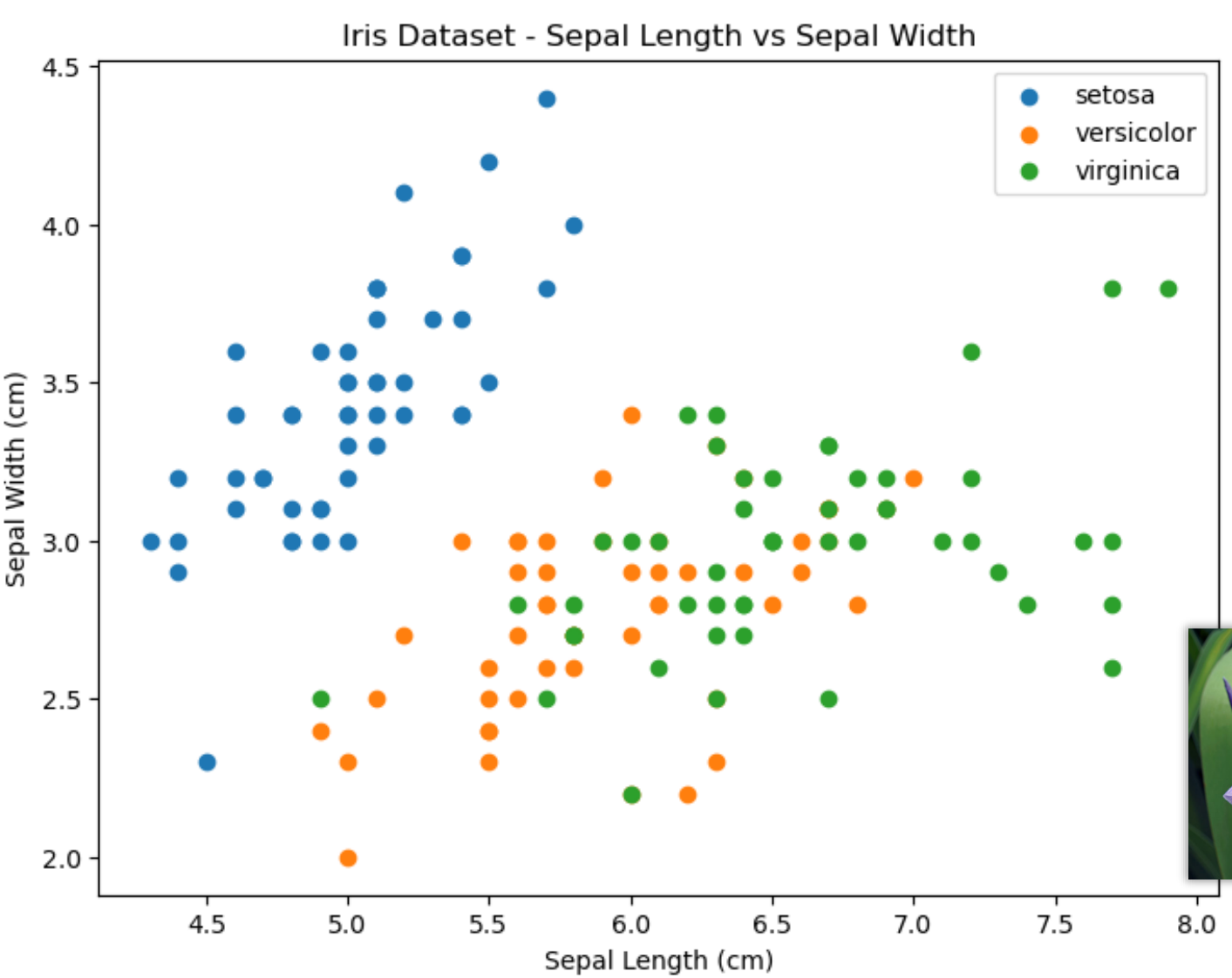
Test your knowledge



1. What does the hyperparameter “k” in kNN control?
2. Why is feature scaling important for kNN?
3. Is kNN a parametric or non-parametric model?
4. What happens if k = 1?
5. How does KNN make predictions?
6. What are the advantages of kNN?
7. What are the disadvantages of kNN?
8. How do we train a kNN model in python?

AD NYE VEJE
AALBORG UNIVERSITET

Let’s get familiar in Python

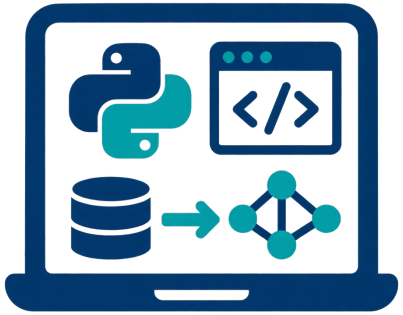








Exercise



- Import the Iris dataset from the Moodle .csv file
- Check the dataset for missing values. Are there any rows or columns that need to be removed?
- Visualise the data using two selected features
- Split the dataset into a training set and a test set
- Train a kNN model using the two selected features and the training set
- Evaluate the model on the unseen test data and calculate the accuracy
- Visualise the model's decision boundary using the code provided in *VisualizationCode.pdf* on Moodle. Does the decision boundary look reasonable, or does the model appear to overfit?
- Try different values of k. How does changing k affect model performance?
- Optional: Repeat the same procedure but with a Naive Bayes classifier and compare its performance with the kNN model

AD NYE VEJE
AALBORG UNIVERSITET